Exercício 1 - Hora de usar as setas

Utilize o que foi aprendido neste capítulo e refatore a função `imprimeProduto` para utilizar arrow functions. Não deixe de customizar o método para utilizar templates strings.

```
let carrinho = [
    { nome: 'abacaxi', preco: '2.00' },
    { nome: 'detergente', preco: '2.50' },
    { nome: 'bolacha', preco: '3.80' }
]

carrinho.forEach(function(produto) {
    imprimeProduto(produto.nome, produto.preco);
})

function imprimeProduto(nome, preco) {
    console.log('Produto: ' + nome + ' | ' + 'Preço: ' + preco);
}
```

Exercício 2 - Vou lavar sua boca com sabão!

Utilize as arrow funcions para deixar o código a seguir ainda menor.

```
let palavroes = [
    "Inconstitucionalíssimo",
    "Otorrinolaringologista",
    "Pneumoultramicroscopicossilicovulcanoconiose",
    "Oftalmotorrinolaringologista"
];

let tamanhos =  palavroes.map(function(palavrao){ return palavrao.length }

console.log(tamanhos); // [ 22, 22, 44, 28 ]
```

Exercício 3 - Não são só umas reticências?

Qual a diferença básica entre os operadores `Rest` e `Spread`?

Exercício 4 - A união faz a força

Refatore o código a seguir para utilizar o operador `Spread` ao invés do método `concat`.

```
const equipeMarketing = ['Joana', 'Marcela', 'Bruna'];
const equipeComercial = ['Talita', 'Luisa', 'Vitória'];

const timeCompleto = equipeMarketing.concat(equipeComercial);

realizaBrainstorm(timeCompleto);
```

Exercício 5 - Pegando a propriedade na lata

Refatore o trecho de código a seguir para utilizar a técnica de desestruturamento.

```
const email = usuario.email;
const nome = usuario.nome;
const idade = usuario.idade
```

Exercício 6 - Cara-Crachá

Otimize o trecho de código a seguir utilizando o desestruturamento.

```
const profissional = {
  titulo: 'Engenheiro de Software',
  departamento: 'Engenharia'
};

function isEngenheiro(profissional) {
  const titulo = profissional.titulo;
  const departamento = profissional.departamento;

  return titulo.indexOf("Engenheiro") > -1 && departamento === 'Engenharia
}

isEngenheiro(profissional); // true
profissional.titulo = 'Marketing';
isEngenheiro(profissional); // false
```

Exercício 7 - O meu videogame é muito melhor que o seu

Aproveitando a classe `VideoGame` criada no exercício anterior, implemente a classe `PlayStation` que contém todas as propriedades do `VideoGame` mais as seguintes proprieades:

- nEntradasUSB: inteiro que representa a quantidade de entradas usb do console
- voltagem: inteiro que representa a voltagem do console.
- adicionais: array de strings que representam os nomes dos adicionais do videogame (ex: adicionais=['Controle sem fio', 'Volante'])

Não esqueça de invocar um objeto da classe.

Exercício 8 - Você cumpre as suas promessas?

Implemente o método simulaPromise() que recebe um parâmetro chamado `sucesso` que é um Boolean que indicará se a Promise foi realizada com sucesso ou não. O método deve receber esta parâmetro e imprimir a mensagem: `ok` em caso de sucesso e `not ok` no caso contrário.

Exemplo:

```
simulaPromise(false); // not ok
simulaPromise(true); // ok
```

Não se esqueça de utilizar o `resolve`, `reject`, `then` e `catch` para resolver o exercício.

Exercício 9 - Você cumpre as suas promessas em tempo?

Para melhorar a simulação de Promise criada no exercício anterior, faça com o método receba mais um argumento chamado `delay` que é um inteiro que representa os milisegundos que a solicitação demora para responder. Ao executar o método, ele deve demorar o tempo do `delay` para exibir a mensagem no console.

Exemplo:

```
simulaPromise(true, 2000); // ok
simulaPromise(false, 1000); // not ok

// ordem que aparece no console
// not ok
// ok
```

Exercício 10 - Passando promessa de pai para filho

Qual é o problema com o trecho de código a seguir? E como consertá-lo?

```
// ...
promise
  .then((data) => console.log(`resultado positivo: ${data}`))
  .then((data) => console.log(`resultado positivo 2: ${data}`))
  .catch((data) => console.log(`resultado negativo: ${data}`));
```